

RefNest

软件架构与开发说明文档

Software Architecture & Developer Reference

Version 0.1.0

Electron 36 • React 18 • TypeScript 5.8 • better-sqlite3

本文档描述 RefNest 桌面端文献管理应用的技术架构、目录结构、核心模块设计、IPC 通信协议以及开发工作流，供开发者参考。

Last updated: 2026-07-07

Contents

1	项目概述 (Project Overview)	2
1.1	技术栈	2
2	目录结构 (Directory Layout)	2
3	进程架构 (Process Architecture)	3
3.1	主进程 (Main Process)	3
3.2	预加载脚本 (Preload)	4
3.3	渲染进程 (Renderer)	4
4	自定义协议 (refnest-file://)	4
5	数据库层 (Database Layer)	4
5.1	引擎与配置	4
5.2	表结构	5
5.3	imagedir 附件类型	5
6	IPC 通信协议	5
6.1	业务通道清单	5
7	PDF 转 Markdown 系统	7
7.1	整体流程	7
7.2	Agent API（免费模式）	7
7.3	Precision API（精准模式）	7
7.4	串行队列	8
8	Markdown 查看器	8
8.1	渲染管线	8
8.2	LocalImage 组件（图片 IPC Blob URL 模式）	8
9	图片画廊 (ImageGalleryPane)	8
10	关键词提取系统	9
10.1	提取优先级	9
10.2	分词逻辑 (splitKeywords)	9
11	UI 组件与样式规范	9
11.1	布局结构	9
11.2	视觉规范	10
11.3	状态管理	10
12	本地 HTTP 服务器	10
13	设置系统	10

14 开发工作流	10
14.1 环境要求	10
14.2 常用命令	11
14.3 Git 分支策略	11
14.4 构建产物	11
14.5 路径别名（Vite 配置）	11
15 已知注意事项与陷阱	11
16 待扩展功能 (Future Work)	12

1 项目概述 (Project Overview)

RefNest（包名 `refnest`）是一款基于 Electron 36 的跨平台桌面端文献管理应用，面向科研人员设计，核心功能包括：

- PDF 导入、元数据自动抓取（CrossRef API）
- AI 驱动的 PDF → Markdown 转换（MinerU API，支持免费/精准两种模式）
- 多模态 Markdown 查看（含图片、公式、表格渲染）
- 图片画廊（精准解析后自动提取）
- 文献标签管理与关键词自动提取
- BibTeX / CSL-JSON 导入导出
- 层级文件夹（集合）管理
- 本地 HTTP 服务器（端口 23120），供浏览器扩展使用

1.1 技术栈

层次	技术	版本
桌面壳	Electron	36.3.x
构建工具	electron-vite	3.1.x
前端框架	React	18.3.x
状态管理	Zustand	5.0.x
类型系统	TypeScript	5.8.x
数据库	better-sqlite3	11.9.x
打包工具	electron-builder	26.x
样式	Tailwind CSS	4.x
国际化	i18next	24.x
Markdown	react-markdown + remark/rehype	10.x
数学公式	KaTeX	0.17.x
PDF 渲染	pdf.js	4.9.x
PDF 操作	pdf-lib	1.17.x
ZIP 处理	adm-zip	0.5.x
引文格式化	citeproc	2.4.x

2 目录结构 (Directory Layout)

RefAI/	
├── src/	
│ ├── main/	# Electron 主进程 (Node.js)
│ │ ├── index.ts	# 入口：窗口创建、协议注册、菜单
│ │ ├── ipc.ts	# 所有 IPC 处理器注册
│ │ └── db/	# SQLite 数据层
│ │ ├── index.ts	# 初始化、迁移运行器
│ │ ├── items.ts	# 文献 CRUD + FTS5 搜索
│ │ ├── attachments.ts	# 附件 CRUD (含 imagedir 类型)
│ │ └── collections.ts	# 集合 CRUD

├── creators.ts	# 作者/编者 CRUD
├── tags.ts	# 标签 CRUD + 孤儿清理
├── mineruApi.ts	# MinerU API 客户端 (Agent + Precision)
├── pdf2mdService.ts	# 串行转换队列 + 设置管理
├── pdfImporter.ts	# PDF 导入 + 关键词提取
├── crossref.ts	# CrossRef DOI 查询
├── importer.ts	# BibTeX / CSL-JSON 导入
├── server.ts	# 本地 HTTP 服务器 (端口 23120)
├── preload/	
│ └── index.ts	# contextBridge -- window.refnest API 桥接
├── renderer/	
│ ├── index.html	
│ └── src/	
│ ├── App.tsx	# 根组件: Dialog 路由
│ ├── main.tsx	# React 挂载点
│ ├── components/	
│ ├── layout/	# MainLayout, Toolbar, StatusBar
│ ├── item-tree/	# CollectionPane, ItemListPane
│ ├── detail-panel/	# DetailPane, MetadataTab,
│ └── AttachmentsTab, TagsTab	
│ ├── pdf-viewer/	# PdfViewer, MarkdownViewer,
│ └── ImageGalleryPane, PdfAnnotationViewer	
│ └── tools/	# Pdf2mdDialog, SettingsDialog, ToolsDialog
│ ├── stores/	# Zustand 状态仓库
│ ├── i18n/	# 国际化资源 (zh/en)
│ └── stubs/	# Node built-in 的浏览器端桩
├── shared/	
│ └── types.ts	# 主/渲染进程共享类型
├── browser-extension/	# Chrome 浏览器扩展 (与本地服务器通信)
├── resources/	# 应用图标
├── electron.vite.config.ts	# Vite 构建配置
├── package.json	# 依赖声明
├── readme/	
│ ├── architecture/	# 本文档所在目录
│ └── analysis/	# 架构对比分析文档

3 进程架构 (Process Architecture)

Electron 应用由三个相互隔离的进程组成，通过严格定义的通道通信。

3.1 主进程 (Main Process)

运行在 Node.js 环境，拥有完整文件系统和操作系统访问权限。src/main/index.ts 在 app.whenReady() 中顺序完成：

1. 注册 refnest-file:// 自定义协议（见 §4）
2. initDatabase() – 初始化 SQLite，执行迁移
3. startLocalServer() – 启动本地 HTTP 服务器（端口 23120）
4. registerIpcHandlers(ipcMain) – 挂载全部 IPC 通道
5. buildMenu(locale) – 构建本地化原生菜单

6. createWindow() – 创建 BrowserWindow

3.2 预加载脚本 (Preload)

`src/preload/index.ts` 在渲染进程加载前注入。通过 `contextBridge.exposeInMainWorld` 将两个全局对象暴露给渲染进程：

- `window.electron` – 标准 `@electron-toolkit/preload` API
- `window.refnest` – 完整的 RefNest 业务 API（见 §6）

安全约束: `contextIsolation: true, nodeIntegration: false, sandbox: false`（需要 `sandbox:false` 以允许 preload 访问 `ipcRenderer`）。

3.3 渲染进程 (Renderer)

标准 React 18 SPA，通过 Vite 构建，运行于 Chromium 渲染器中。状态管理使用 Zustand，无 Redux；路由内嵌于组件逻辑，无 React Router。

4 自定义协议 (refnest-file://)

Electron 渲染器中 `file://` 协议受 CSP 限制，无法直接访问本地文件。RefNest 注册了 `refnest-file://` 作为特权协议：

```
1 protocol.registerSchemesAsPrivileged([
2   { scheme: 'refnest-file',
3     privileges: { secure: true, supportFetchAPI: true, stream: true } }
4 ])
5
6 // Handler (Windows path fix):
7 protocol.handle('refnest-file', (request) => {
8   let filePath = decodeURIComponent(
9     request.url.replace('refnest-file://', ''))
10  // refnest-file:///C:/path -> strip leading /
11  if (/^\\[A-Za-z]:/.test(filePath)) filePath = filePath.slice(1)
12  return net.fetch(pathToFileURL(filePath).toString())
13 })
```

注意：Markdown 图片的实际显示通过 IPC blob URL 路径实现（见 §8），不依赖此协议，从而规避了所有 CSP 限制。

5 数据库层 (Database Layer)

5.1 引擎与配置

使用 `better-sqlite3`（原生 Node 模块，同步 API）。`src/main/db/index.ts` 初始化时设置：

```
1 PRAGMA journal_mode = WAL;    -- 写前日志，提升并发读性能
2 PRAGMA foreign_keys = ON;     -- 强制外键约束
```

数据库文件位于 Electron `userData` 目录（Windows: `%APPDATA%/refnest`）。

5.2 表结构

表名	说明
libraries	单行种子表, id=1 "My Library"
items	文献主表 (key, type, title, abstract, year, doi, url, journal, publisher, volume, issue, pages, isbn, language, extra, deleted, library_id, timestamps, version)
creators	作者实体表 (first_name, last_name, orcid)
item_creators	文献-作者关联 (role: author/editor/translator, position)
tags	标签实体 (name UNIQUE)
item_tags	文献-标签关联 (复合主键)
attachments	附件表 (type: pdf/link/other/ imagedir , filename, path, url, mime_type, size, md5)
notes	笔记 (content, timestamps)
collections	层级集合 (parent_id 自引用外键)
collection_items	集合-文献关联
items_fts	FTS5 全文搜索虚拟表 (索引 title + abstract)
sync_state	键值同步状态存储

5.3 imagedir 附件类型

精准解析 (Precision API) 产生的图片目录以 `type='imagedir'` 存入 `attachments`, 字段 `path` 存储目录绝对路径。删除时跳过 `unlinkSync` (目录由用户管理)。

6 IPC 通信协议

所有渲染进程-主进程通信通过 `ipcRenderer.invoke(channel, ...args)/ipcMain.handle(channel, handler)` 模式实现, 返回 `Promise`。推送事件使用 `ipcMain.send+ipcRenderer.on`。

6.1 业务通道清单

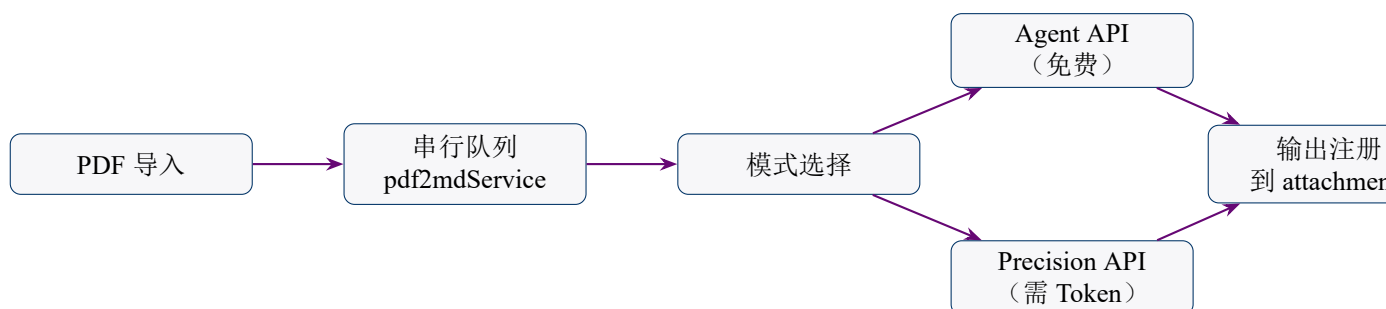
通道	功能
文献 (items)	
items:getAll	获取全部文献 (含标签)
items:getTrashed	获取回收站文献
items:getByCollection	按集合筛选
items:getById	按 id 获取单条
items:create	新建文献
items:update	更新字段
items:trash	软删除 (moved to trash)
items:restore	从回收站恢复
items:delete	永久删除
items:emptyTrash	清空回收站
items:search	FTS5 全文搜索

<code>items:extractKeywords</code>	从 MD/PDF 提取关键词并写入标签
作者 (creators)	
<code>creators:getByItem</code>	获取文献作者列表
<code>creators:setForItem</code>	批量替换文献作者
标签 (tags)	
<code>tags:getByItem</code>	获取文献标签
<code>tags:getAll</code>	获取全部标签
<code>tags:setForItem</code>	批量设置文献标签（自动清理孤儿）
集合 (collections)	
<code>collections:getAll</code>	获取集合树
<code>collections:create</code>	新建集合
<code>collections:rename</code>	重命名
<code>collections:delete</code>	删除集合
<code>collections:addItem</code>	添加文献到集合
<code>collections:removeItem</code>	从集合移除文献
<code>collections:getItems</code>	获取集合内文献 id 列表
附件 (attachments)	
<code>attachments:getByItem</code>	获取文献附件列表
<code>attachments:add</code>	文件选择对话框 + 注册附件
<code>attachments:remove</code>	删除附件记录（ <code>imagedir</code> 不删文件）
<code>attachments:getPath</code>	获取附件绝对路径
<code>attachments:openExternally</code>	用系统默认程序打开
<code>attachments:openPath</code>	用 shell 打开（资源管理器定位）
导入 (import)	
<code>import:openDialog</code>	多文件选择，分发到 PDF/BibTeX/CSL-JSON 导入器
文件系统 (fs)	
<code>fs:readFile</code>	读取文件为 <code>number[]</code> （绕过 <code>contextBridge</code> 限制）
<code>fs:readTextFile</code>	读取文本文件（UTF-8）
<code>fs:writeFile</code>	写入文件
<code>fs:listDir</code>	递归列出目录内所有图片文件路径
<code>pdfjs:workerPath</code>	返回 <code>pdf.js worker</code> 绝对路径
工具 (tools)	
<code>tool:pick-pdf</code>	选择 PDF 文件路径
<code>tool:pick-dir</code>	选择输出目录
<code>tool:pdf2md</code>	手动触发 PDF 转换（流式进度回调）
设置 (settings)	
<code>settings:get</code>	读取键值设置
<code>settings:set</code>	写入键值设置
<code>settings:pickStoragePath</code>	选择文件存储根目录
推送事件（主进程 → 渲染进程）	
<code>pdf2md:status</code>	转换队列状态更新（ <code>filename</code> , <code>state</code> , <code>message</code> , <code>pending</code> ）

<code>tools:open</code>	原生菜单触发打开工具面板
<code>settings:open</code>	原生菜单触发打开设置面板
<code>settings:setLocale</code>	原生菜单切换语言

7 PDF 转 Markdown 系统

7.1 整体流程



7.2 Agent API（免费模式）

端点: <https://mineru.net/api/v1/agent/parse>

1. 读取 PDF 页数 (`pdf-lib`)
2. 若页数 > 20, 用 `pdf-lib` 拆分为 ≤ 20 页的分块
3. 逐块: POST 获取上传地址 $\rightarrow$ PUT 上传 $\rightarrow$ 轮询结果 (每 3 秒, 最多 6 分钟)
4. 合并所有分块 Markdown, 以 --- 分隔, 写入 `<stem>.md`
5. 注册为文献附件 (`registerAttachment`)

输出: 单一 .md 文件, 无图片。

7.3 Precision API（精准模式）

端点: <https://mineru.net/api/v4>, 需要 Bearer Token。

1. POST `/file-urls/batch`: 获取 OSS 预签名上传 URL + `batch_id`
2. PUT 上传整个 PDF (不分块)
3. 轮询 `/extract-results/batch/{batch_id}` (每 5 秒, 最多 20 分钟)
4. 下载结果 ZIP, 用 `adm-zip` 解压到 `<stem>_mineru/`
5. 递归查找 `full.md` (`findFile`)
6. 查找 `full.md` 同级目录下的图片子目录
7. 注册 `full.md` 为 Markdown 附件, 图片目录为 `imagedir` 附件

关键设计: `zip.extractAllTo(extractDir, true)` 保留 ZIP 内部原始目录结构, 使 `full.md` 中的相对图片路径自然有效, 无需路径重写。

7.4 串行队列

pdf2mdService.ts 维护一个 QueueItem[] 数组, drainQueue() 通过 _running 标志确保同一时刻只有一个转换任务运行。每个任务完成/失败后通过 pdf2md:status 推送事件通知渲染进程。

8 Markdown 查看器

文件: src/renderer/src/components/pdf-viewer/MarkdownViewer.tsx

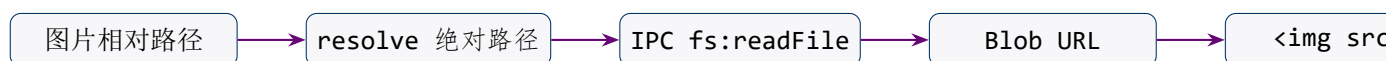
8.1 渲染管线

```
1 <ReactMarkdown
2   remarkPlugins={[remarkGfm, remarkMath]}
3   rehypePlugins={[rehypeKatex, rehypeRaw]}
4   components={{ img: LocalImage, ... }}
5 >
6   {markdown}
7 </ReactMarkdown>
```

- remark-gfm: 表格、删除线、任务列表等 GitHub Flavored Markdown 扩展
- remark-math: 解析 $...$ 和 $...$ 数学块
- rehype-katex: 将数学块渲染为 KaTeX HTML
- rehype-raw: 透传 MinerU 输出中的原始 HTML 标签

8.2 LocalImage 组件 (图片 IPC Blob URL 模式)

由于渲染进程无法直接访问 file://, 图片通过以下路径加载:



Windows 路径注意事项:

- path-browserify 是 POSIX 实现, dirname('H: path file') 返回 '.'。解决方案: dirname(filePath.replace(/\\/g, '/'))
- isAbsolute('C:/path') 返回 false, 需自定义检测: /^[A-Za-z]:\\/.test(n)

9 图片画廊 (ImageGalleryPane)

文件: src/renderer/src/components/pdf-viewer/ImageGalleryPane.tsx
精准解析完成后, imagedir 附件出现在附件面板, 点击即可打开图片画廊。

- 调用 fs:listDir IPC 递归枚举目录内所有图片文件 (扩展名: .png .jpg .jpeg .gif .webp .svg .bmp)

- 网格布局: `grid-template-columns: repeat(auto-fill, minmax(160px, 1fr))`
- 缩略图通过 **Thumb** 组件, 同样使用 **IPC Blob URL** 模式加载
- 点击缩略图打开 **Lightbox**: 全屏显示, 支持键盘 `←/→` 切换、**Esc** 关闭

10 关键词提取系统

文件: `src/main/pdfImporter.ts`

10.1 提取优先级

1. **Markdown** 附件 (MinerU 输出结构更清晰, 优先读取):
 - **Bold** 内联: `**Keywords**`: `word1; word2`
 - **Plain** 内联: `Keywords`: `word1; word2`
 - **Heading** 块: `## Keywords`\n `word1; word2`
 - 同时支持中文: 关键词
2. **PDF** 原始文本 (`pdf-parse-new`, 仅前 8 页): 回退方案
3. **CrossRef subject** 字段: 导入时自动合并

10.2 分词逻辑 (`splitKeywords`)

优先级依次: `•` (中圆点) > `;` (分号) > `,` (逗号) > 多空格。
结果过滤: 长度 2–80 字符, 最多 20 个关键词。

11 UI 组件与样式规范

11.1 布局结构

左 侧 边 栏 (220px)	中 央 内 容 区 (flex:1)	右 侧 详 情 (320px)
CollectionPane	ItemListPane (文献列表) – 或 – PdfReaderPane / MarkdownReaderPane / ImageGalleryPane	DetailPane

中央区同一时刻只渲染一个: 文献列表或某种查看器 (PDF/Markdown/图片画廊)。
查看器打开时右侧详情面板隐藏 (`viewerPath !== null` 时不渲染 `DetailPane`)。

11.2 视觉规范

元素	字体	字号 / 颜色
文献条目标题	Times New Roman, Georgia, serif	18px
标签气泡	Adobe Gothic Std B, Source Han Sans, Microsoft YaHei	14px / #660874
标签背景	rgba(102,8,116,0.07), 边框 rgba(102,8,116,0.22)	
界面文本	系统 sans-serif	13px

11.3 状态管理

三个 Zustand Store:

- **itemStore**: 当前选中文献、文献列表、查看器路径/类型 (`viewerType: 'pdf' | 'markdown' | 'gallery'`)、集合筛选、搜索关键词、年份排序
- **collectionStore**: 集合树数据
- **statusStore**: 状态栏队列指示、pdf2md 转换进度

12 本地 HTTP 服务器

监听 **127.0.0.1:23120**, 为浏览器扩展提供本地 API, 支持从浏览器直接向 RefNest 导入当前页面的文献元数据。

浏览器扩展位于 **browser-extension/**, 通过 **background.js** 与本地服务器通信。

13 设置系统

设置以 JSON 文件存储于 Electron **userData** 目录下的 **refnest-settings.json**, 通过 **settings:get/set** IPC 通道读写。

键名	说明
tool.pdf2md.enabled	是否启用 PDF 自动转换 (默认 true)
tool.pdf2md.mode	'agent' (免费) 或 'precision' (精准)
tool.pdf2md.apiToken	Precision API Bearer Token
storage.path	用户指定的文件存储根目录

14 开发工作流

14.1 环境要求

- Node.js ≥ 20 , npm ≥ 10
- Windows 11 (主要开发平台); macOS / Linux 理论可用
- **better-sqlite3** 需要原生编译: **npm run postinstall** 自动执行

14.2 常用命令

```
npm install          # 安装依赖 (含 better-sqlite3 原生编译)
npm run dev          # 开发模式 (electron-vite dev, 热重载)
npm run build        # 构建 (输出到 out/)
npm run package      # 打包安装程序 (输出到 dist/)
npm run typecheck    # TypeScript 类型检查
npm run lint         # ESLint 检查
npm run test         # vitest 单元测试
```

14.3 Git 分支策略

- 所有 RefAI 功能提交必须推送到 feat/pdf-annotation 分支，严禁直接提交到 master/main。
- 功能开发完成后通过 PR 合并。

14.4 构建产物

目录	内容
out/main/	主进程编译输出 (由 electron-vite 生成)
out/preload/	预加载脚本编译输出
out/renderer/	渲染进程编译输出 (Vite 打包)
dist/	electron-builder 打包的安装程序

14.5 路径别名 (Vite 配置)

```
'@'          -> src/renderer/src
'@shared'    -> src/shared
'fs'         -> src/renderer/src/stubs/fs.ts // 渲染进程 Node fs 桩
'path'       -> path-browserify             // POSIX 实现 (注意 Windows 路径问题)
```

15 已知注意事项与陷阱

1. **path-browserify Windows 路径问题:** dirname 和 isAbsolute 对 C:\... 路径行为异常。使用前必须将反斜杠替换为正斜杠: `p.replace(/\\/g, '/')`, 并用自定义函数检测绝对路径。
2. **contextBridge 传递二进制数据:** contextBridge 不能透传 Buffer, fs.readFile 返回 number[], 渲染进程用 new Uint8Array(bytes) 重建再创建 Blob。
3. **better-sqlite3 重建:** Electron 版本升级后必须重新执行 npm run rebuild, 否则会出现 NODE_MODULE_VERSION 不匹配错误。
4. **MinerU ZIP 结构:** full.md 的确切位置 (根目录 vs 子目录) 依 MinerU 版本而异, findFile() 递归查找。图片子目录紧邻 full.md 所在目录, 名称不固定, 当前取第一个子目录。

5. **Precision API 超时:** Precision API 接受完整 PDF（无需分块），但超时上限设为 20 分钟（240 次 × 5 秒）。大文件（>200 页）可能超时。
6. **imagedir 删除行为:** 删除 imagedir 类型附件仅从数据库移除记录，不删除磁盘上的图片目录，防止误删用户文件。

16 待扩展功能 (Future Work)

- Zotero 同步协议集成（sync_state 表已预留）
- 注释层持久化（PdfAnnotationViewer 框架已存在）
- 免费模式（Agent API）图片支持（当前无图片输出）
- 引文插入（citeproc 依赖已引入）
- 网页存档（浏览器扩展已具备基础框架）
- 多窗口 / 浮动阅读器